# Spanner Access-Pattern Guide

*Engineering Handbook · Internal · Owner: Platform Engineering*

## Purpose

Cloud Spanner gives us horizontal scale and strong consistency, but only if we shape reads and transactions to match how it stores and locks data. This page is the prescriptive how-to for teams whose services (api-service, worker-service, ingestion-service) read and write Spanner. It exists to stop two recurring failure modes: reads that time out with `DEADLINE_EXCEEDED` under peak load (BUG-1023), and read-write transactions that fail with `ABORTED` from lock contention (BUG-1024). Follow it before shipping any new query or transaction.

## Standards

### Design queries against an index, not a scan

Every hot read path must be backed by an index that covers its filter and, where possible, its projection. A query that filters on a non-key column with no matching index forces a full-table scan whose latency grows with the table — the root cause of BUG-1023. Add a secondary index, and add a `STORING` clause so the read is covered and never joins back to the base table.

```
-- Covering index: filter on status, return id + created_at without a base-table lookup
CREATE INDEX ItemsByStatus
 ON Items(status)
 STORING (created_at, owner_id);
```

Always select only the columns you need, bound the result set with a `LIMIT`, and inspect the plan with `EXPLAIN` / query stats (`SPANNER_SYS.QUERY_STATS_TOP_MINUTE`) before merging. Reject any plan showing a full scan on a large table.

### Set deadlines deliberately — and use stale reads

The client default deadline is short; raise it *moderately* for legitimately heavy reads instead of masking a bad plan. For dashboards, lists, and other non-critical paths, use a bounded-staleness read: it can be served from any replica and cuts tail latency dramatically.

```
from google.cloud import spanner
from google.cloud.spanner_v1 import RequestOptions

# Non-critical read: tolerate ~15s staleness, served from a nearby replica
with database.snapshot(exact_staleness=datetime.timedelta(seconds=15)) as snap:
   rows = snap.execute_sql(
       "SELECT id, created_at FROM Items@{FORCE_INDEX=ItemsByStatus} "
       "WHERE status = @s LIMIT 100",
       params={"s": "ACTIVE"}, param_types={"s": spanner.param_types.STRING},
       timeout=8.0,  # explicit, moderate RPC deadline
   )
```

### Keep read-write transactions short and ordered

Spanner uses pessimistic locking for read-write transactions and *aborts* one side of a conflict to break contention (BUG-1024). Minimise the blast radius: read only the rows you will write, do slow work (external calls, CPU) *outside* the transaction, and always acquire/write rows in a consistent key order so two transactions never lock in opposite order. Let the client library retry aborts — the `run_in_transaction` helper does this automatically, so the callable must be idempotent and free of side effects.

```
def _apply(txn):
   # read only what we write; consistent order by primary key
   row = txn.execute_sql(
       "SELECT balance FROM Accounts WHERE id=@id",
       params={"id": account_id}, param_types={"id": spanner.param_types.STRING},
   ).one()
   txn.update("Accounts", ["id", "balance"], [[account_id, row[0] + delta]])

database.run_in_transaction(_apply)  # auto-retries ABORTED
```

Avoid hot spots: never use a monotonically increasing key (timestamp, sequence) as the leading primary-key column — hash or UUID-prefix it so writes spread across splits.

## Common failures and fixes

- `DEADLINE_EXCEEDED` on reads at peak → add a covering/secondary index, narrow the projection + `LIMIT`, raise the deadline moderately, and move non-critical reads to bounded-staleness snapshots (BUG-1023).
- Transactions failing with `ABORTED` under concurrency → shorten the transaction, read only what you write, order writes consistently, and rely on the library's automatic retry (BUG-1024).
- Latency climbs with table size → you are scanning; confirm with query stats and add the missing index.
- Write latency spikes on one split → monotonic leading key causing a hot spot; re-key with a hashed/UUID prefix.

## Checklist

- Every hot read is backed by an index covering its filter (and projection via `STORING`).
- Query plan inspected with `EXPLAIN`/query stats; no full scans on large tables.
- Explicit, moderate RPC deadline set on each read.
- Non-critical reads use bounded-staleness or exact-staleness snapshots.
- Read-write transactions are short, read only what they write, and write in a consistent key order.
- Transaction callables are idempotent so automatic ABORTED retries are safe.
- Primary keys avoid monotonic leading columns to prevent hot spots.

## Related incidents

BUG-1023, BUG-1024